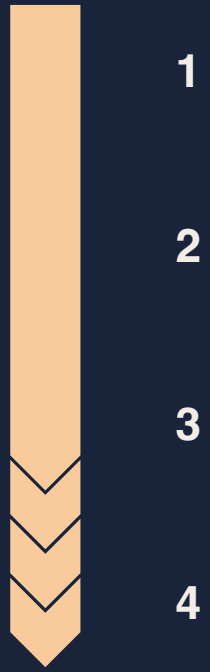
CSE 251B
NanoGPT Competition Presentation

Rhea Senthil Kumar

TABLE OF CONTENTS



Introduction

Team Introduction

Methodology

Data Processing, Engineering Tricks, Model

Experiments

Looped Layers, Distillation

Discussion

Reflections, Learnings, and Future Work

Introduction

Team Introduction

1

Team Introduction

Originally from **Maryland!**



MIT Lincoln Laboratory

- Worked in Space Systems and Analysis
- High performance computing for signal cross-correlation (IEEE HPEC)
- MIT BWSI Build a CubeSat Challenge for high school students!

Undergrad @ UC Berkeley

- High-energy astrophysics and ML research at Space Science Lab (IEEE NSS)
- NASA GSFC Intern
- Fellowship building anti-particle detector

M.S. @ UC San Diego

- Gravitational Wave Paleontology Group
- Diffusion/Flow Matching for Star Population Synthesis and Mergers
- Developing Generative AI Course for MIT BWSI!

More Computational Astrophysics!

Problem Statement

The Problem:

Train a GPT-style language model and achieve lowest possible perplexity

- <=100M parameters
- <=5 min. inference

How I Solved It:

- Preliminary literature review, data processing experiments, ablations on smaller models + early training trends
- Training tricks: distillation, flash attn, gradient accumulation
- Modern architecture choices: Llama-style

Note: Throughout presentation, the



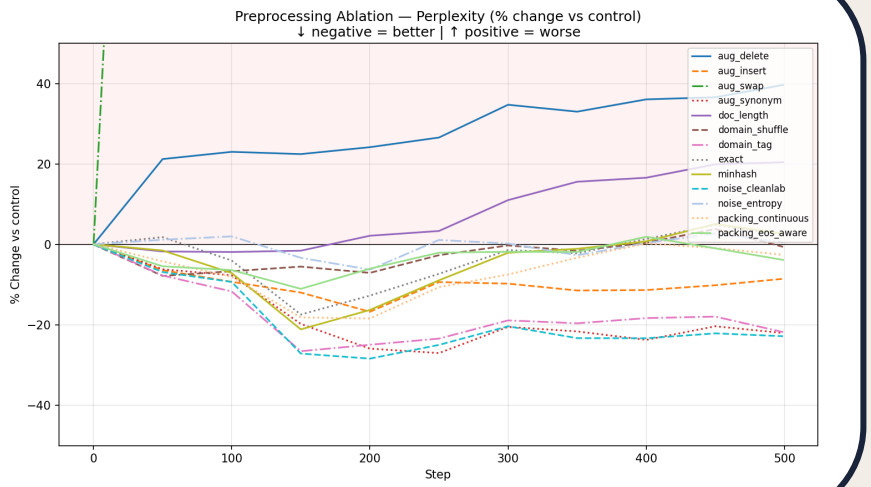
emoji represents a key learning referenced in the discussion section.

2

Methodology

Data Processing, Model, Engineering Tricks

Data Processing & Engineering Tricks



Used and augmented FineWeb-Edu 10B dataset:

- Replace words with their synonyms
- Randomly insert tokens/noise into sequences
- Combine short sequences together and cut off at EOS
- Pack sequences continuously to learn transitions between them

Used 3B total tokens for training (Chinchilla scaling) on a NVIDIA GeForce GTX 4090 GPU.

Distillation: GPT2 and GPT-Neo, used as “teacher models” after pretraining to finetune model

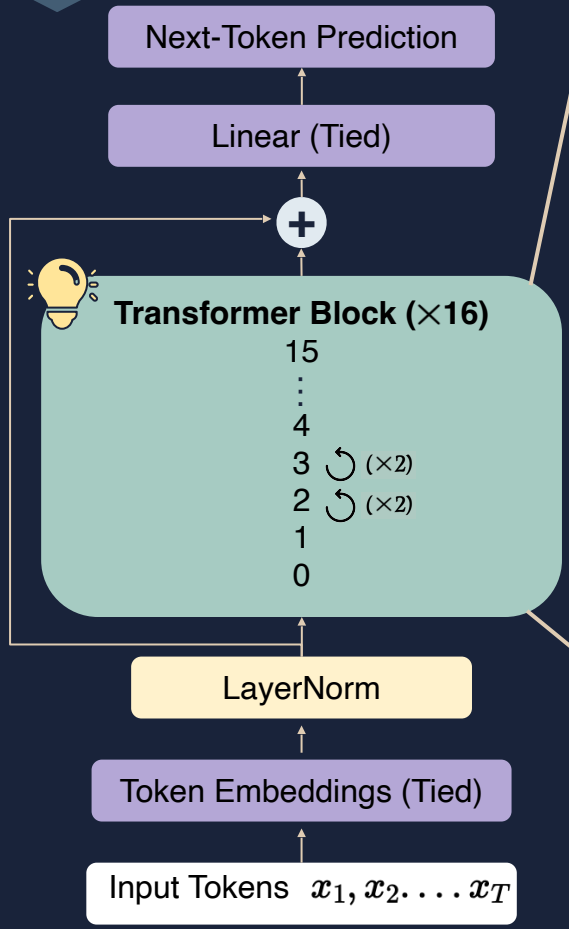
Flash Attention: Optimize scaled dot-product attention for faster training { $O(N^2) \rightarrow O(N)$ }

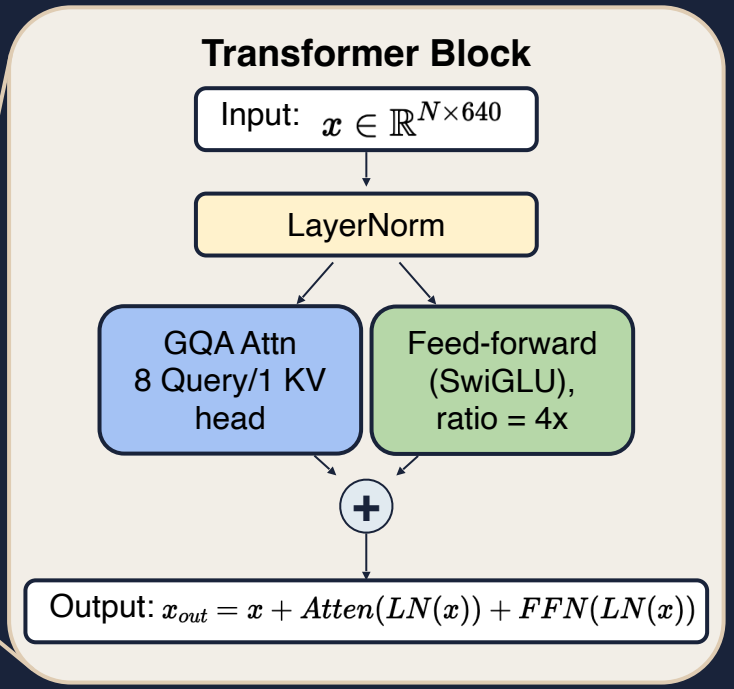
Context Curriculum: Pretrain on smaller context length (256), increase to 1024 to finetune

Gradient Accumulation: Gain larger effective batch size by adding gradients over multiple batches

Weight Tying: Save parameters by letting embedding and output matrix share weights

Deep Learning Model





Component	Parameters (Approx.)
Token Embedding (tied)	50,257*640 = 32.3M
Attention over 16 Layers	~26.9M
FFN over 16 Layers	~39.4M
LayerNorm scales (2/layer + 1 final)	~21K
QK Gain (1/layer)	16
TOTAL	**99,365,888**

Attention (GQA)
- 8 Query heads, 1 Key-Value head
- Scaled Dot-Product Attention (SDPA/Flash)
- QK-Norm with learned gain
- Logit soft-capping

Feed-Forward Network
- SwiGLU activation
- Muon optimizer
- No bias in linear layers

Positional Encoding
- Rotary Position Embedding (RoPE)

Normalization & Residuals
- LayerNorm (Pre-LN)
- Parallel residual connections
- Residuals start at layer 0
- Scaled residual initialization

Embeddings
- Token embedding tied with output projection (640d)
- No bias terms in linear layers

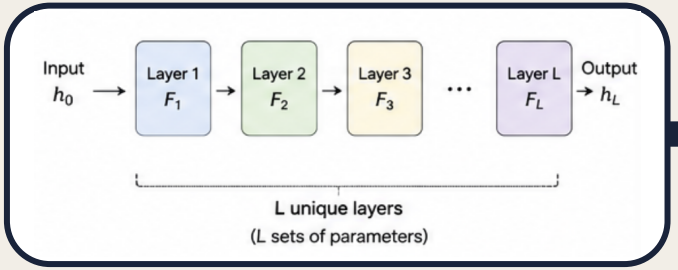
Experiments

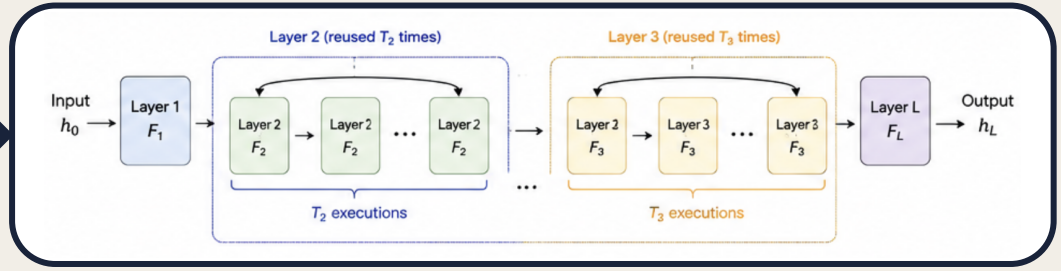
Looped Layers, Distillation

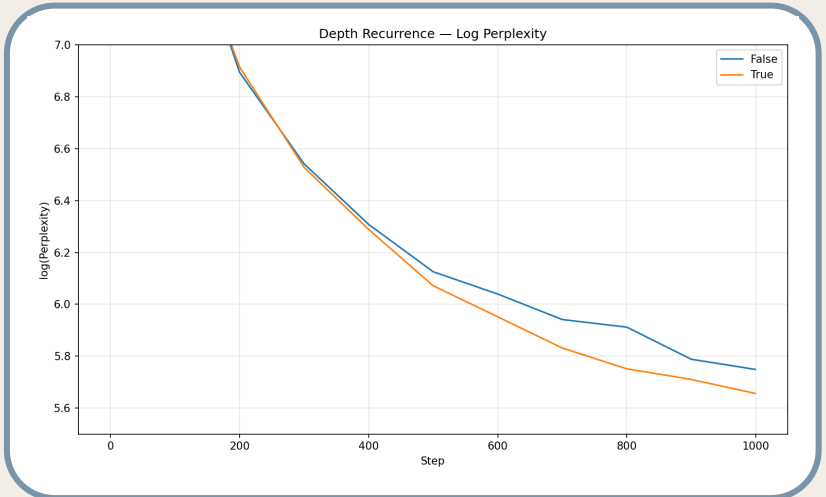
3

Experiment 1: Looped/Recurrent Layers

Looped layers: Apply the same layer to itself repeatedly instead of only learning unique sets of parameters every layer







Fu et al., Fully Looped Transformer (2026)

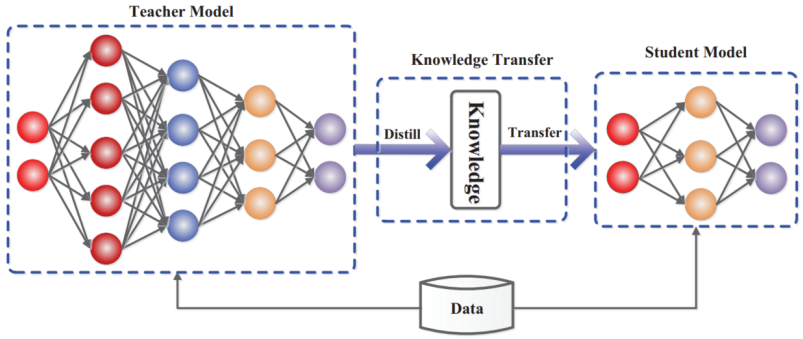
- Looped layers are claimed to produce "additional computation for improved performance without increasing parameter count."
- But training can also becomes unstable as loop iterations increase, due to gradient oscillation and residual explosion

Personal observations:

- The choice of which layers to loop does not affect performance for this task (chose layers 2,3 arbitrarily).
- Looping more than once prohibitively drives up runtime for this task without notable performance gain.

Experiment 2: Knowledge Distillation

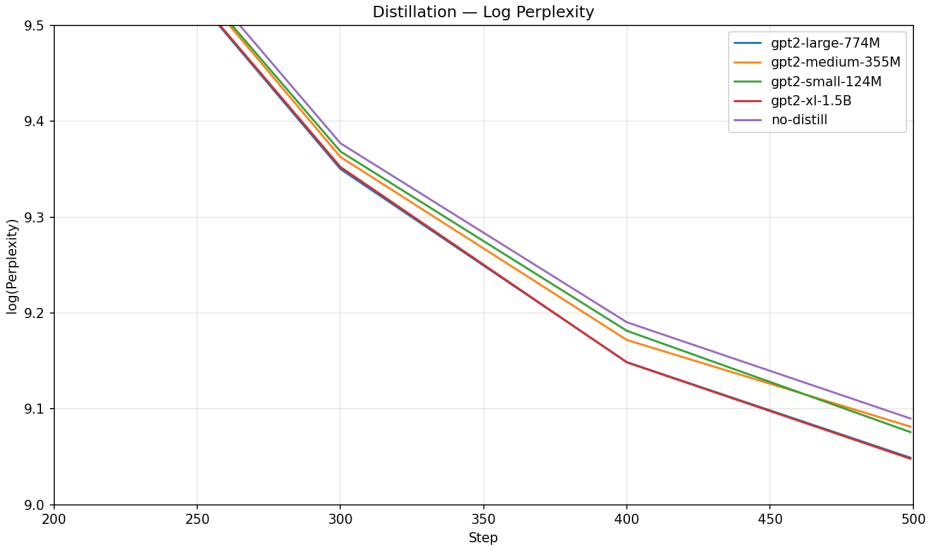
Distillation: Train smaller model/student to learn from predictions of larger model/teacher instead of just the data



Gou et al., Knowledge Distillation: A Survey (2021)

- Cross Entropy and KL Loss
- Cost of larger teachers: more training time due to larger forward pass required
- Performance improves when using distillation during fine-tuning, after initial training.

Comparison of Teacher Models by Size



4

Discussion

Reflections, Learnings, and Future Work

Strengths

- Combination of modern architecture choices and training tricks (SwiGLU, RoPE, distillation, looped layers, flash attention, etc.)
- Effective data pre-processing techniques and augmentations which improved data quality
- Rigorously tested optimization parameters, architecture, and training techniques using automated hyperparameters sweeps (Weights & Biases) to get optimal combinations

Weaknesses

- Model size inherently limits perplexity performance, and tied embeddings may limit flexibility of model
- Weight tying may have hurt model's ability to represent language
- Dependence on FineWebEdu dataset and aggressive processing, may not generalize to other datasets

Limitations

- Bottlenecked by compute-resources (GPUS), machine memory, sequential experiments
- Quality of teacher model (distillation) directly impacts the extent to which model can learn.
- Time-limited: model is extremely undertrained given time and compute constraints.

Learnings: For some training options, better performance in early training may not extend to later training.

Data Pre-Processing

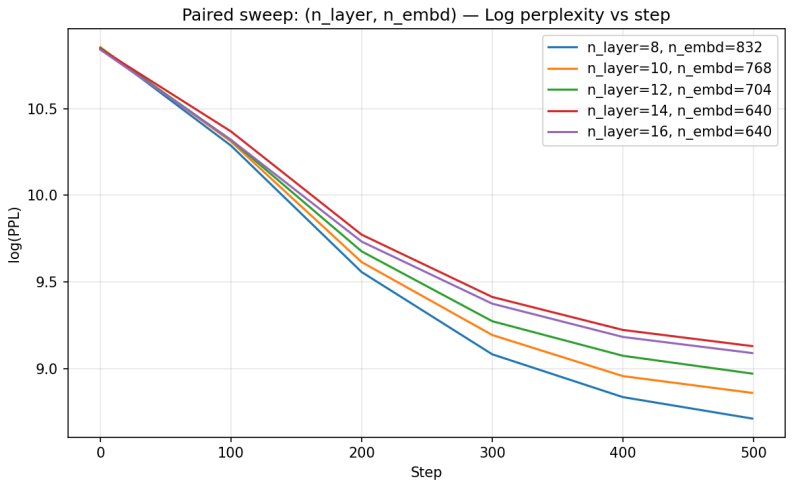
- In early training, significantly helps perplexity (1414 PPL vs. 2181 in 100 steps)
- However in later training, model plateaus at ~25 validation PPL.
- **Why?** Early on roughly matching distribution of data (so training/val loss drops fast) but in fine tuning trying to match exactly. Techniques like synonym swapping, random insertion etc. will cause more noise/ harder to match distribution.

Additionally, **some training options don't extend well from smaller models to bigger models**, and must be tuned on the actual test model size. This limited the time I had for experimentation.

- LR/LR schedule, choosing recurrence layers, attention-specific parameters, etc.

Number of Layers

- Ablations during early training suggested fewer layers performed better
- Hence, focused on an 8-layer model for much of development
- Eventually tried 16 layers, achieving lower perplexity in later stage training (~25 PPL)
- **Why?** Smaller models are easier to optimize so they have lower loss in early training, but have a capacity ceiling. The pattern switches over time since more layers means model has better representational capacity.



What I'd Do Next

- Redo training options/hyperparameters with longer ablations
- Find minimum embedding dimension and from there make deepest model
- Use a bigger batch size (memory constraints limited to 128, increase to 1024 or 2048)
- Train for more steps
- Start with aggressive data pre-processing to speed up early learning, then gradually reduce over time so the model is exposed to the true data distribution
- Perform one continuous training run instead of multiple start/stops

Sample Q&A

- What new information did you learn about transformers from the project that you didn't know from class?
- If you could start over, what would you change about your process?
- What's one approach you thought would work but didn't work in your architecture?
- What training technique would you want to experiment with next?
- Which architecture change from your ablations gave you the best improvement in perplexity?

THANK YOU!

Do you have any questions?

CREDITS: *This presentation template was created by , and includes icons by , and infographics & images by*

Please keep this slide for attribution